

IDO6001 Homework 04

장서현 (학번: 2025324096)

Part 1: Mathematical Formulation

Problem 1.1

Model Formulation using Completion Time Variables

($1 \mid prec \mid \sum w_j T_j$)의 경우, 목적함수($\sum w_j T_j$)와 제약 조건이 모두 완료시간(C_j)을 매개로 정의되므로, 완료시간 변수를 사용하는 것이 가장 직관적이고 간결하다고 판단했다. 특히 Precedence Constraints (job $i \rightarrow j$)의 경우, $C_j \geq C_i + p_j$ 형태의 단순한 선형 부등식만으로 처리할 수 있어 구현이 효율적이다.

Notation

Sets and Indices

- $N = \{1, 2, \dots, n\}$: Set of jobs
- $i, j \in N$: Indices for jobs
- $prec$: Set of precedence pairs (i, j) where job i must precede job j

Parameters

- p_j : Processing time of job j
- d_j : Due date of job j
- w_j : Weight of job j
- M : A sufficiently large number (Big-M)

Decision Variables

- C_j : Completion time of job j
- T_j : Tardiness of job j
- $y_{ij} = \begin{cases} 1, & \text{if job } i \text{ precedes job } j \\ 0, & \text{otherwise} \end{cases}$
: Binary variable defined for unordered pairs $((i, j), (j, i) \notin prec \text{ and } i < j)$

Mathematical Model

$$\begin{aligned} \text{Minimize} \quad & \sum_{j \in N} w_j T_j & (1a) \\ \text{s.t.} \quad & T_j \geq C_j - d_j & \forall j \in N & (1b) \\ & T_j \geq 0 & \forall j \in N & (1c) \\ & C_j \geq p_j & \forall j \in N & (1d) \\ & C_j \geq C_i + p_j & \forall (i, j) \in prec & (1e) \\ & C_j \geq C_i + p_j - M(1 - y_{ij}) & \forall i, j \in N, i < j, (i, j), (j, i) \notin prec & (1f) \\ & C_i \geq C_j + p_i - M y_{ij} & \forall i, j \in N, i < j, (i, j), (j, i) \notin prec & (1g) \\ & C_j \geq 0 & \forall j \in N & (1h) \\ & y_{ij} \in \{0, 1\} & \forall i, j \in N, i < j, (i, j), (j, i) \notin prec & (1i) \end{aligned}$$

Constraints

- (1b) ~ (1c): Tardiness Constraints
- (1d): Basic Completion Time Constraints
- (1e): Precedence Constraints for pairs (i, j) defined in the precedence set
- (1f) ~ (1g): Disjunctive Constraints for unordered pairs (i, j) where no precedence relationship exists
- (1h) ~ (1i): Variable Domains

Code Implementation

- Single machine with precedence.ipynb 코드 및 결과 첨부

Problem 1.2

Model Formulation using Assignment and Positional Date Variables

$(1 \mid s_{ij} \mid \sum w_j T_j)$ 의 경우, Sequence-dependent setup time s_{ij} 이 작업 i 바로 뒤에 j 가 오는 경우에만 발생하므로, “즉시 인접” 관계를 명확히 표현할 수 있어야 한다. Assignment/Position 변수는 각 작업의 순서를 위치로 직접 배정하여, 인접한 두 위치 사이에서 setup time을 반영할 수 있다고 판단했다.

Notation

Sets and Indices

- $N = \{1, 2, \dots, n\}$: Set of jobs
- $K = \{1, 2, \dots, n\}$: Set of positions
- $i, j \in N$: Indices for jobs
- $k \in K$: Index for positions

Parameters

- p_j : Processing time of job j
- d_j : Due date of job j
- w_j : Weight of job j
- s_{ij} : setup time if job i precedes job j
- M : A sufficiently large number (Big-M)

Decision Variables

- $u_{jk} = \begin{cases} 1, & \text{if job } j \text{ is assigned at position } k \\ 0, & \text{otherwise} \end{cases}$
- γ_k : Completion time of position k
- T_j : Tardiness of job
- $z_{ijk} = \begin{cases} 1, & \text{if job } i \text{ is at pos } k-1 \text{ and job } j \text{ is at pos } k \\ 0, & \text{otherwise} \end{cases}$

Mathematical Model

$$\text{Minimize } \sum_{j \in N} w_j T_j \quad (2a)$$

$$\text{s.t. } \sum_{k \in K} u_{jk} = 1 \quad \forall j \in N \quad (2b)$$

$$\sum_{j \in N} u_{jk} = 1 \quad \forall k \in K \quad (2c)$$

$$\gamma_k \geq \gamma_{k-1} + \sum_{j \in N} p_j u_{jk} + \sum_{i \in N} \sum_{j \in N, j \neq i} s_{ij} z_{ijk} \quad \forall k \in K \setminus \{1\} \quad (2d)$$

$$z_{ijk} \geq u_{i,k-1} + u_{jk} - 1, \quad z_{ijk} \leq u_{i,k-1}, \quad z_{ijk} \leq u_{jk} \quad \forall i, j \in N, \forall k \in K \setminus \{1\} \quad (2e)$$

$$\gamma_1 \geq \sum_{j \in N} (s_{0j} + p_j) u_{j1} \quad (2f)$$

$$T_j \geq \gamma_k - d_j - M(1 - u_{jk}) \quad \forall j \in N, \forall k \in K \quad (2g)$$

$$T_j \geq 0 \quad \forall j \in N \quad (2h)$$

$$u_{jk}, z_{ijk} \in \{0, 1\} \quad \forall i, j \in N, \forall k \in K \quad (2i)$$

$$\gamma_k \geq 0 \quad \forall k \in K \quad (2j)$$

Constraints

- (2b): Each job is assigned to exactly one position
- (2c): Each position is filled by exactly one job
- (2d): Calculates the completion time at position k (γ_k)
- (2e): (Linearization Constraints) Forces $z_{ijk} = 1$ only if both $u_{i,k-1} = 1$ and $u_{jk} = 1$
- (2f): (Assuming s_{0j} is the initial setup time) First Position Constraint
- (2g) ~ (2h): Tardiness Constraints
- (2i) ~ (2j): Variable Domains

Code Implementation

- Single machine with setup time.ipynb 코드 및 결과 첨부

Part 2: Simulation

Problem 2.1

```
1 def select_job(queue, rule, t):
2     if not queue:
3         return None
4
5     elif rule == "CR":
6         return min(queue, key=lambda x: (x.due_date - t) / x.total_remaining_time if x.total_remaining_time > 0 else 0)
7
8     elif rule == "EDD":
9         return min(queue, key=lambda x: x.due_date)
10
11    elif rule == "MST":
12        # Minimum Slack Time: 여유 시간(Slack)이 가장 적은 작업 우선
13        # Slack = 납기일(due_date) - 현재시간(t) - 남은 총 가공시간(total_remaining_time)
14        # 값이 작을수록(음수 포함) 여유가 없어 긴급한 작업
15        return min(queue, key=lambda x: x.due_date - t - x.total_remaining_time)
16
17    elif rule == "MDD":
18        # Modified Due Date: 변경된 납기일이 가장 빠른 작업 우선
19        # MDD = max(원래 납기일, 현재시간 + 남은 총 가공시간)
20        # 제시기에 끝낼 수 있으면 '납기일', 이미 늦을 것 같으면 '가장 빠른 완료 예정 시간' 기준
21        return min(queue, key=lambda x: max(x.due_date, t + x.total_remaining_time))
```

Figure 1: MST/MDD 구현 코드

문제에서 제시한대로 machine 1에 MST 규칙, machine 2에 MDD 규칙을 적용하면 아래와 같은 스케줄을 얻을 수 있다.

```
1 machine1 = Machine(env = env, number = 1, rule = 'MST')
2 machine2 = Machine(env = env, number = 2, rule = 'MDD')
```

Figure 2: M1-MST, M2-MDD 적용 코드

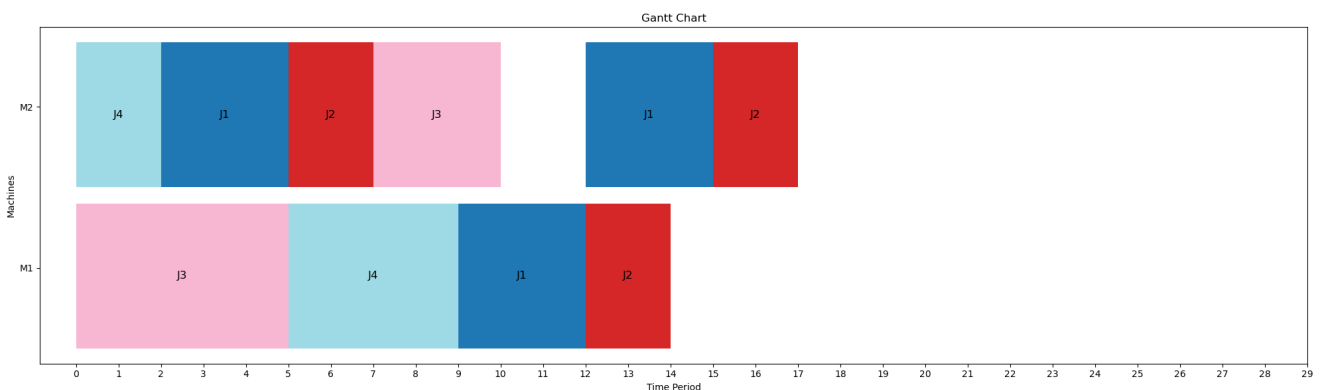


Figure 3: M1-MST, M2-MDD 적용 스케줄

같은 기계를 공유하는 작업 간에는 납기가 급한 순서(J4→J1→J2)로 처리가 이루어졌으나, Job 1은 긴 공정 경로와 최우선 순위 작업(J4) 대기로 인해 후반부에 완료되었으며, Job 3은 비경합 경로(M₁ 시작)를 통해 납기 여유에도 불구하고 조기 완료되는 경로상의 이점을 보였다.

Problem 2.2

CR, EDD, MST, MDD 규칙을 두 머신에 조합하여 총 12개의 시뮬레이션을 수행한 결과, 규칙조합마다 다른 스케줄을 도출하였지만, 아래와 같이 실질적으로 유사한 성능지표가 도출됨을 확인할 수 있다. (각 머신에 적용한 규칙을 (M1, M2)로 표현)

- Case 1: (CR, EDD), (CR, MST), (CR, MDD), (EDD, CR), (EDD, MST), (EDD, MDD), (MST, EDD), (MST, MDD)

Makespan	Mean Completion Time	Mean Flow Time	Max Flow Time	Mean Lateness Time
17	12.75	12.75	17	1.25
Max Lateness Time	Mean Tardiness Time	Max Tardiness Time	Number of Overdue	
5	2.75	5	3	

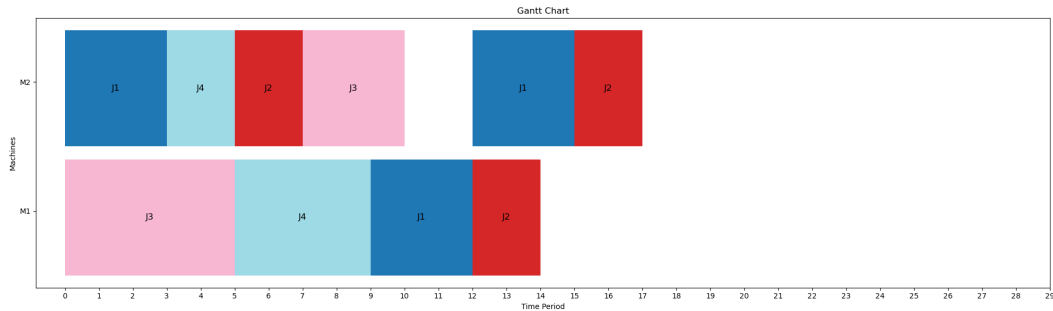


Figure 4: (M1-EDD, M2-MST) 적용 스케줄 및 성능지표

- Case 2: (MDD, CR), (MDD, EDD), (MDD, MST)

Makespan	Mean Completion Time	Mean Flow Time	Max Flow Time	Mean Lateness Time
17	12.75	12.75	17	0.75
Max Lateness Time	Mean Tardiness Time	Max Tardiness Time	Number of Overdue	
7	2.25	7	3	

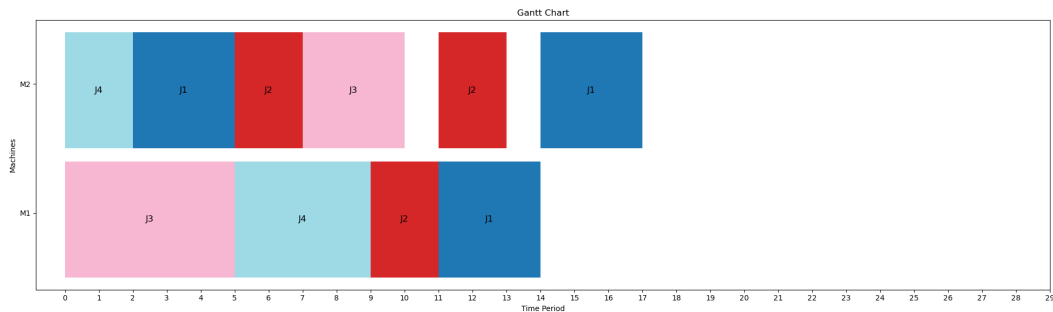


Figure 5: (M1-MDD, M2-EDD) 적용 스케줄 및 성능지표

- Case 3: (MST, CR)

Makespan	Mean Completion Time	Mean Flow Time	Max Flow Time	Mean Lateness Time
16	12.75	12.75	16	1.25
Max Lateness Time	Mean Tardiness Time	Max Tardiness Time	Number of Overdue	
4	2.75	4	3	

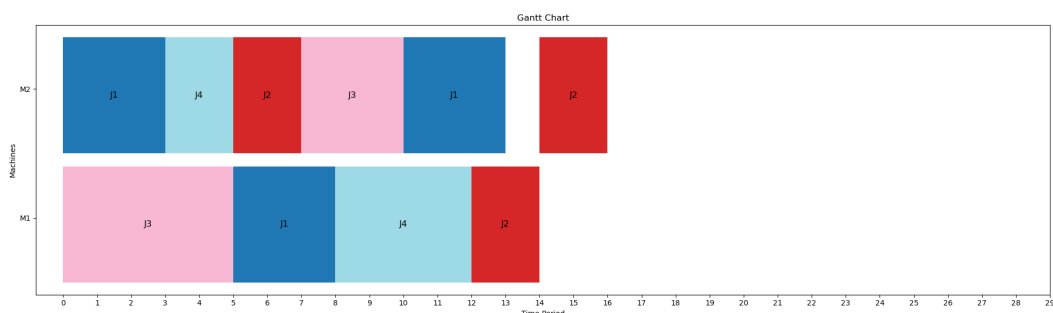


Figure 6: (M1-MST, M2-CR) 적용 스케줄 및 성능지표

Remark 1.

Case 1과 Case 2는 거의 유사한 성능지표 값이 도출되었지만, Case 3 (MST, CR) 조합만이 Makespan 16, Max Tardiness 4라는 유의미하게 다른 결과를 도출하였다. (EDD, CR)이나 (MDD, CR)과 같이 M2에 동일하게 CR을 적용한 다른 Case들이 최적해를 찾지 못한 점을 고려할 때, 성능 향상의 원인은 M2의 CR 규칙과 M1의 MST 규칙 간의 시너지 효과에 기인한다.

Remark 2.

M2는 Job 1, 2, 4가 진입하는 초기 병목 지점이다. EDD나 MDD는 단순히 납기가 빠른 Job 4($d_4 = 8$)를 우선시하여, 전체 공정 경로가 가장 긴 Job 1($d_1 = 10$, 3단계 공정)의 투입을 지연시킨다. 반면, CR 규칙은 '남은 총 가공 시간'을 고려하므로 작업 부하가 큰 Job 1을 가장 긴급한 작업으로 식별하여 M2에 우선 투입한다. 이는 Job 1이 후속 공정인 M1으로 빠르게 이동할 수 있는 전제 조건을 형성한다.

Remark 3.

M2에서 CR 규칙에 의해 Job 1이 조기 투입되더라도, M1에서 적절한 스케줄링이 이루어지지 않으면 병목 현상이 재발한다. M1에 적용된 MST 규칙은 납기뿐만 아니라 현재 시점의 slack을 기준으로 판단한다. Job 1은 납기가 Job 4보다 늦더라도 남은 공정이 많아 slack이 매우 부족한 상태다. MST는 이러한 '실질적 긴급도'를 감지하여, Job 1과 Job 4 사이의 경합을 효율적으로 중재하고 전체 흐름이 막히지 않도록 제어하였다.

```

1  elif rule == "SPT": # Shortest Processing Time
2      def get_current_pt(job):
3          return job.processing_time[job.current_process_step]
4      # 가공 시간이 가장 작은 작업을 선택
5      return min(queue, key=get_current_pt)
6
7  elif rule == "WMOR": # Weighted Most Operations Remaining
8      def calculate_score(job):
9          # 가중치 정의: '남은 공정 수'를 가중치로 설정
10         custom_weight = len(job.processing_path)
11         # Score = (남은 가공 시간) * (남은 공정 수)
12         score = job.total_remaining_time * custom_weight
13         return score
14         # 점수가 가장 높은 작업을 선택
15         return max(queue, key=calculate_score)
16
17  machine1 = Machine(env = env, number = 1, rule = "SPT")
18  machine2 = Machine(env = env, number = 2, rule = "WMOR")

```

앞선 분석을 통해 M2(초기병목)에서 작업 부하가 큰 J1과 J2를 초기에 투입하는 것이 Makespan 단축에 기여함을 확인하였다.

이를 바탕으로 다음과 같은 디스패칭 전략을 수립하였다.

- WMOR (M2): (잔여 가공시간 × 잔여 공정 수)를 가중치로 부여하는 규칙 WMOR를 고안하여 작업부하가 큰 작업(J1, J2)에게 높은 우선순위 부여
- SPT (M1): 처리 시간이 짧은 작업을 우선 처리함으로써 시스템의 흐름을 가속화

Figure 7: (M1-SPT, M2-WMOR) 디스패칭 규칙 정의 코드

[결론] 제안한 (M1-SPT, M2-WMOR) 전략은 Makespan을 15로 단축하며 기존 Case 3보다 뛰어난 효율성을 달성하였다. 이는 M2의 WMOR 규칙이 작업 부하가 큰 핵심 작업을 초기에 투입하여 시스템 병목을 선제적으로 해소했기 때문이다. 비록 처리 효율을 극대화하는 과정에서 최대 지연 시간은 소폭 증가했으나, 전체 지연 작업 수는 오히려 감소하였다.

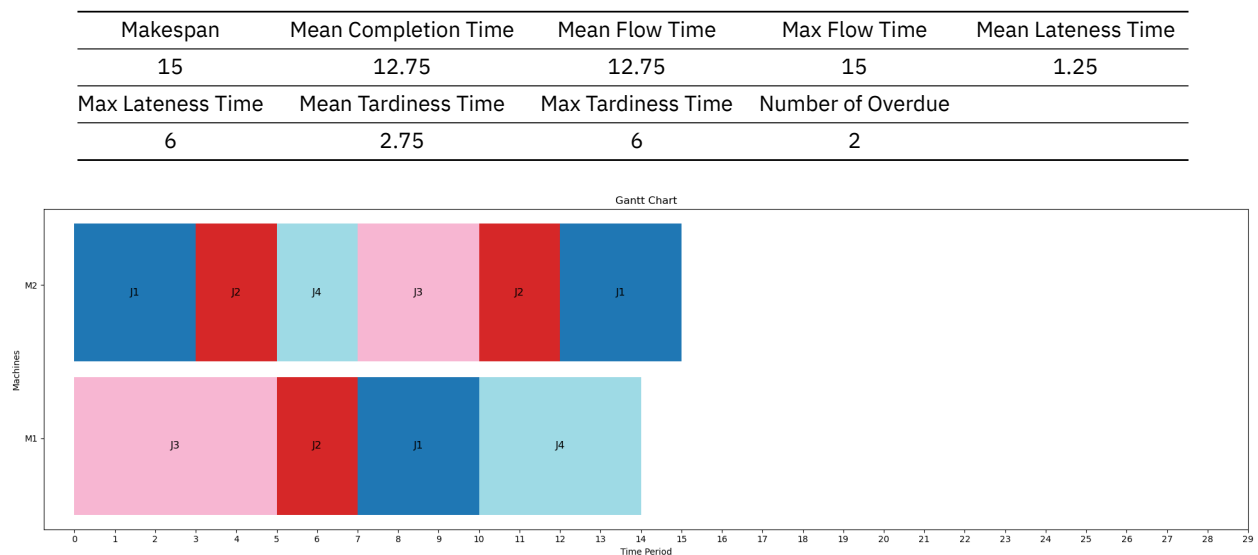


Figure 8: (M1-SPT, M2-WMOR) 적용 스케줄 및 성능지표